BATTLESHIP MULTIPLAYER

BÁO CÁO GIỮA KỲ

Môn Lập trình mạng căn bản (NT106) · UIT

Hoàng · Đức · Dũng

Mục Lục Nội Dung

Giới Thiệu

Cơ Sở Lý Thuyết

Phân Tích Và Thiết Kế

Danh Mục Từ Viết Tắt

Viết tắt	Ý nghĩa
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
NAT	Network Address Translation
JSON	JavaScript Object Notation
DTO	Data Transfer Object
AES	Advanced Encryption Standard
CBC	Cipher Block Chaining
IV	Initialization Vector

Viết tắt	Ý nghĩa
ORM	Object-Relational Mapping
EF Core	Entity Framework Core
ERD	Entity Relationship Diagram
REST	Representational State Transfer
RLS	Row Level Security
UI	User Interface
GDI+	Graphics Device Interface Plus
NT106	Lập trình mạng căn bản

Giới Thiệu Đề Tài

Battleship (Bắn tàu) nhiều người chơi qua mạng

- Hai người chơi, mỗi người một lưới 10×10 và một đội tàu.
- Hai bên **lần lượt** gọi tọa độ để bắn; mỗi phát bắn là **trúng** hoặc **hụt**.
- Ai đánh chìm hết tàu đối phương trước thì thắng.
- Vị trí tàu của đối phương luôn bị **che** (sương mù).

Vì sao chọn đề tài này cho NT106

Trò chơi theo lượt, dữ liệu nhỏ, nhưng đủ các bài toán mạng thật: kết nối, đồng bộ trạng thái, chống gian lận, ngắt kết nối giữa chừng.

[CHƯA CÓ ẢNH]

`images/gk-01-man-hinh-tran.png`

Màn hình bàn cờ 10x10 của game (chụp ở Client). Nếu chưa có gameplay thì chụp lưới 10x10.

Bối Cảnh Và Vấn Đề Cần Giải Quyết

Bối cảnh

- Game cờ/bắn tàu trực tuyến rất phổ biến; nhiều game dùng sẵn thư viện thời gian thực (SignalR, Socket.IO, Photon...).
- Các thư viện này **che giấu tầng socket**, sinh viên không thấy được cách dữ liệu thật sự đi qua mạng.

Bài toán nhóm đặt ra

- Tự xây **giao thức trên TCP thô**: đóng gói, bắt tay, định tuyến gói tin.
- **Server nắm luật**: Client không được tự quyết Hit/Miss hay nhìn trộm bàn đối thủ.
- Xử lý **nhiều người chơi đồng thời**, ngắt kết nối đột ngột.
- **Bảo mật**: mã hóa đường truyền, mật khẩu băm, chống dò mật khẩu.
- Dữ liệu dùng chung qua **cơ sở dữ liệu trên cloud**.

Phạm Vi Đề Tài

Trong phạm vi

- Tài khoản: đăng ký, đăng nhập, trạng thái Online/Offline.
- Sảnh chờ: chat, danh sách phòng, tạo/vào phòng bằng ID.
- Ghép trận tự động (Matchmaking).
- Xếp tàu, bắn theo lượt, sương mù, xử thắng khi đối thủ mất kết nối.
- Lưu lịch sử trận, bảng xếp hạng.
- Bảo mật: AES-128, BCrypt, khóa đăng nhập.
- Tính năng đặc biệt: **Radar** và **gửi kết quả trận ra Discord/Telegram**.

Ngoài phạm vi

- Chế độ khán giả (Spectator).
- Đối thủ máy (AI), bản di động, đồ họa 3D.
- Thanh toán, hệ thống bạn bè nâng cao.

Nền tảng

Ứng dụng desktop **Windows** (WinForms, C# .NET 8). Chạy trên máy cá nhân; demo giữa hai mạng khác nhau bằng Ngrok.

Đối Tượng Sử Dụng Và Đầu Ra Mong Đợi

Đối tượng sử dụng

- **Người chơi**: chơi trận, chat, xem bảng xếp hạng.
- **Người vận hành** (nhóm): chạy Server, quản lý dữ liệu trên Supabase.

Vai trò trong nhóm

Đức	Server, phòng, luật trận
Dũng	Giao diện, cơ sở dữ liệu
Hoàng	Giao thức, mạng, bảo mật

Đầu ra

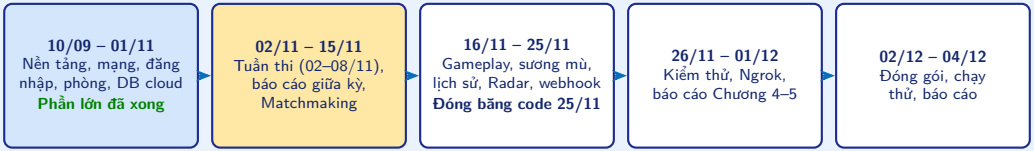
1. **Server** TCP bất đồng bộ (C# .NET 8).
2. **Client** WinForms chơi được từ đăng nhập đến hết trận.
3. **Cơ sở dữ liệu** trên cloud (PostgreSQL) kèm REST đọc bảng xếp hạng.
4. **Giao thức mạng** có tài liệu: khung gói tin, bắt tay, mã hóa.
5. Báo cáo, slide và bản demo.

Tính Năng Và Tiến Độ Giữa Kỳ

Nhóm	Tính năng	Trạng thái
Tài khoản	Đăng ký, đăng nhập (BCrypt, khóa 5 lần sai), trạng thái Online/Offline	**Xong**
Phòng	Tạo/vào/rời phòng bằng ID	**Xong**
Phòng	Kick thành viên (chủ phòng)	**Dở dang**
Ghép trận	Tìm trận ngẫu nhiên (Matchmaking)	**Xong**
Giao tiếp	Chat sảnh	**Xong**
Giao tiếp	Chat trong trận	**Chưa**
Gameplay	Xếp tàu, bắn theo lượt, sương mù, tự xử thắng	**Chưa**
Thống kê	Bảng xếp hạng (REST Supabase)	**Xong**
Thống kê	Lưu lịch sử đấu	**Dở dang**
Bảo mật	Mã hóa AES-128 + BCrypt + giới hạn đăng ký	**Xong**
Dữ liệu	SQLite hoặc PostgreSQL (Supabase) qua EF Core	**Xong**
Đặc biệt	Radar, gửi kết quả ra Discord/Telegram	**Chưa**

Dở dang: đã có mã nhưng chưa nối đủ vào luồng chơi. **Chưa**: bắt đầu sau tuần thi, theo lịch ở slide kế tiếp.

Timeline Cập Nhật Đến Hết Học Kỳ



Báo cáo: **04/12/2026** (dự phòng đến 15/12)

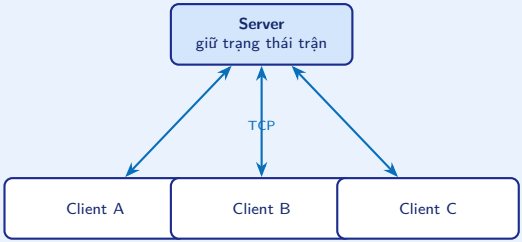
Ghi chú

Việc then chốt là **logic trận đấu trên Server** (nhận Ready, xếp tàu, bắn, kết thúc). Các mục sương mù, lịch sử trận, Radar và webhook đều phụ thuộc vào mục này.

Mô Hình Client–Server Và Giao Thức TCP

Client–Server, Server nắm luật

- Một **Server** trung tâm, nhiều **Client** kết nối vào.
- Server giữ trạng thái trận, Client chỉ gửi **yêu cầu** và hiển thị **kết quả**.
- Nhờ vậy Client không thể tự báo trúng/hụt hay nhìn bàn của đối thủ (chống gian lận).



Vì sao chọn TCP mà không phải UDP

TCP bảo đảm **đến nơi, đúng thứ tự, không mất gói**. Game theo lượt không cần tốc độ cực thấp, nhưng một lệnh bắn bị mất hay đảo thứ tự sẽ làm sai cả ván.

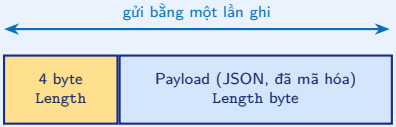
Socket TCP Thô Và Khung Gói Tin (Framing)

Vấn đề của TCP

TCP là **dòng byte liên tục**, không có ranh giới giữa các gói: hai gói có thể **dính** thành một, hoặc một gói bị **vỡ** làm hai lần đọc.

Cách giải: Length-Prefix

Mỗi gói đi trước bởi **4 byte** chứa độ dài (big-endian). Bên nhận đọc đủ 4 byte, rồi đọc đúng độ dài byte, mới giải mã.



Một khung gói tin

- Dùng `TcpListener` / `TcpClient` / `NetworkStream` (`System.Net.Sockets`).
- Server từ chối gói có kích thước không hợp lệ, tránh bị gửi gói rác.

Cách Server phục vụ nhiều người cùng lúc

- `async/await`: chờ đọc/ghi mạng **không chặn luồng**.
- Mỗi Client là **một Task** riêng, một Client chậm không làm kẹt cả Server.
- `ConcurrentDictionary`: danh sách phiên Online an toàn khi nhiều Task cùng sửa.
- `SemaphoreSlim(1,1)`: một Client chỉ có **một Task ghi** vào stream tại một thời điểm.

Rủi ro nếu không khóa

Hai Task ghi xen kẽ byte vào cùng một stream làm **vỡ khung gói**; phía nhận đọc ra độ dài sai và lỗi.

Nhóm cũng gặp lỗi tương tự ở Client khi hai nơi cùng đọc một stream; cách sửa là chỉ **một vòng lặp nền** được đọc stream.

Giao Thức Ứng Dụng: JSON, Bắt Tay, Định Tuyến

Gói tin = `Type` + `Payload`

- `Type`: kiểu gói (`LoginRequest`, `FireShotRequest`...).
- `Payload`: dữ liệu (DTO) tuần tự hóa thành **JSON**.
- JSON dễ đọc, dễ gỡ lỗi; mỗi kiểu gói có một DTO riêng.

Bắt tay (Handshake)

Vừa kết nối, Client gửi **phiên bản giao thức**; Server chấp nhận hoặc ngắt. Tránh Client cũ nói chuyện sai với Server mới.

PacketRouter + IPacketHandler

Mỗi nhóm chức năng là một **handler** đăng ký vào Router theo kiểu gói. Thêm tính năng mới chỉ cần thêm handler, không sửa lõi mạng.

Handler	Xử lý
`AuthHandler`	đăng ký, đăng nhập
`RoomHandler`	phòng đấu
`ChatHandler`	chat sảnh
`MatchmakingHandler`	tìm trận

Bảo Mật: AES-128 Và BCrypt

AES-128-CBC mã hóa đường truyền

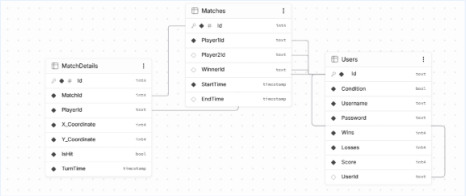
- Mã hóa **đối xứng**: cùng một khóa để mã hóa và giải mã.
- **IV ngẫu nhiên cho mỗi gói**, gửi kèm; cùng nội dung vẫn ra bản mã khác nhau.
- Khóa 16 byte lấy từ `SHA-256` của mật khẩu chung hai phía.

Hạn chế đã biết

Khóa chung nằm trong ứng dụng, chưa trao đổi khóa khi kết nối. Hướng nâng cấp: **TLS** hoặc trao đổi khóa Diffie–Hellman.

BCrypt băm mật khẩu

- Băm **một chiều**, có **salt** ngẫu nhiên, nên hai người cùng mật khẩu vẫn ra hai chuỗi khác nhau.
- Cố tình **chậm** (hệ số chi phí) để kẻ tấn công dò mật khẩu tốn kém.
- Cơ sở dữ liệu **không bao giờ** lưu mật khẩu rõ; chuỗi lưu có dạng `$2a$11$...`



Cơ Sở Dữ Liệu: EF Core, SQLite Và PostgreSQL

Entity Framework Core (ORM, Code-First)

- Viết **lớp C#** (`User`, `Match`, `MatchDetail`), EF tự sinh bảng và câu truy vấn.
- Đổi hệ quản trị chỉ cần đổi **provider**, không viết lại truy vấn.
- Truy vấn qua tham số, chống **SQL injection**.

Hai chế độ chạy

SQLite	file cục bộ, chạy offline, dùng khi phát triển
PostgreSQL (Supabase)	trên cloud, dùng chung cho cả nhóm

Chọn bằng biến môi trường `BATTLESHIP_DB`.

Cloud Và Web Service: Supabase, REST, RLS

Supabase

PostgreSQL trên cloud; tự sinh **REST API** (PostgREST) cho bảng và view. Truy cập qua **HTTPS**.

RLS (Row Level Security)

Quy tắc phân quyền ngay trong cơ sở dữ liệu. Khóa công khai (*publishable key*) chỉ đọc được view `leaderboard`; bảng `Users` (có mật khẩu băm) bị chặn.

Phân tách đường đi của dữ liệu

- **Đăng nhập, chơi**: qua TCP + AES, Server kiểm tra.
- **Bảng xếp hạng**: Client đọc thẳng REST, **chỉ đọc**.

Mật khẩu cơ sở dữ liệu chỉ nằm ở Server (biến môi trường), không đưa vào mã nguồn.

Giao Diện Desktop: WinForms Và GDI+

Vì sao WinForms

- Có sẵn trong .NET 8, thiết kế giao diện bằng kéo thả, chạy ngay trên Windows.
- Bàn cờ 10×10 vẽ bằng **GDI+** (ô cố định, đổi màu theo trúng/hụt).

Lưu ý khi nối mạng

Dữ liệu từ Server về trên **luồng nền**, nhưng chỉ luồng giao diện mới được sửa điều khiển. Phải dùng `Invoke` để tránh lỗi cross-thread.

Công nghệ	Dùng để
C# .NET 8	toàn bộ hệ thống
`System.Net.Sockets`	TCP thô
`System.Text.Json`	đóng gói JSON
AES, BCrypt.Net	mã hóa, băm
EF Core, Npgsql	cơ sở dữ liệu
WinForms, GDI+	giao diện
Supabase	cloud, REST

Yêu Cầu Chức Năng Và Phi Chức Năng

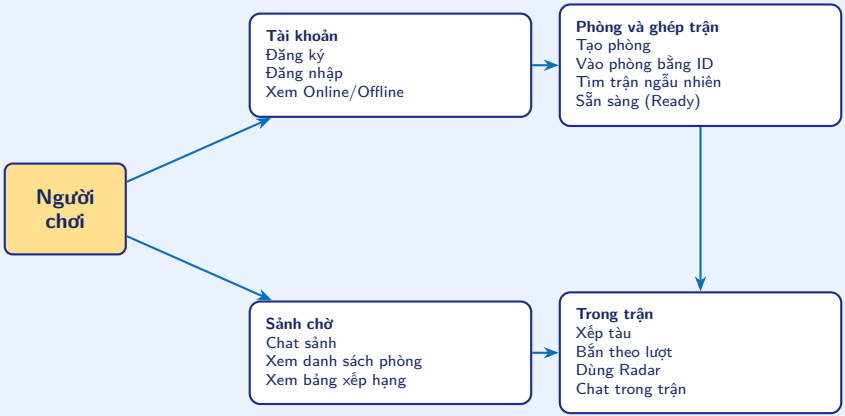
Yêu cầu chức năng

1. Đăng ký, đăng nhập, xem ai đang Online.
2. Chat ở sảnh chờ.
3. Tạo phòng, vào phòng bằng ID, tìm trận ngẫu nhiên.
4. Xếp tàu, sẵn sàng (Ready), bắn theo lượt.
5. Báo trúng/hụt, chìm tàu, kết thúc trận.
6. Lưu lịch sử trận, xem bảng xếp hạng.

Yêu cầu phi chức năng

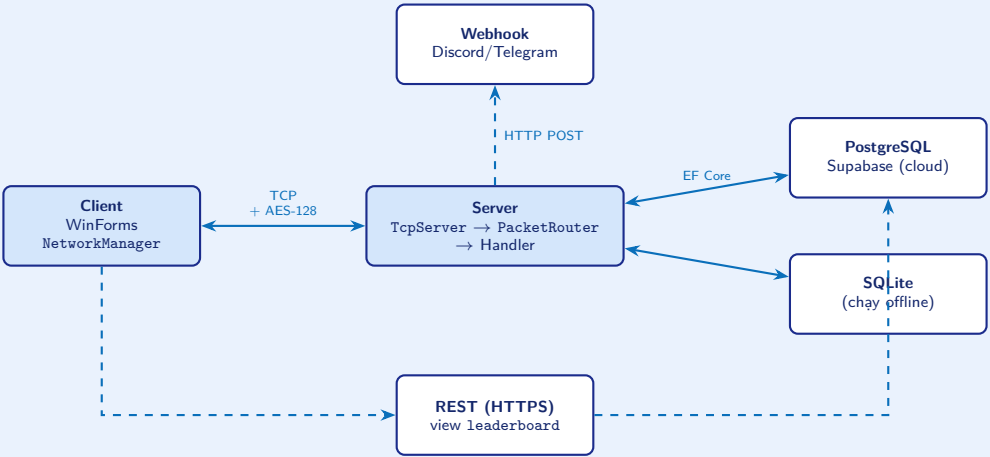
- **Đúng luật**: Server quyết định, chống gian lận.
- **Bảo mật**: mã hóa gói tin, băm mật khẩu, chống dò mật khẩu.
- **Đồng thời**: nhiều người chơi, không nghẽn.
- **Bền**: Client rớt mạng không làm Server sập.
- **Dễ chạy**: có chế độ SQLite offline để demo.

Biểu Đồ Chức Năng Của Người Chơi



Mũi tên giữa các nhóm thể hiện thứ tự dùng. Server là bên xử lý mọi yêu cầu.

Kiến Trúc Tổng Thể



Nét liền: đường chính (TCP, cơ sở dữ liệu). Nét đứt: chỉ đọc bảng xếp hạng qua REST; webhook (đang phát triển).

Thiết Kế Giao Thức Gói Tin

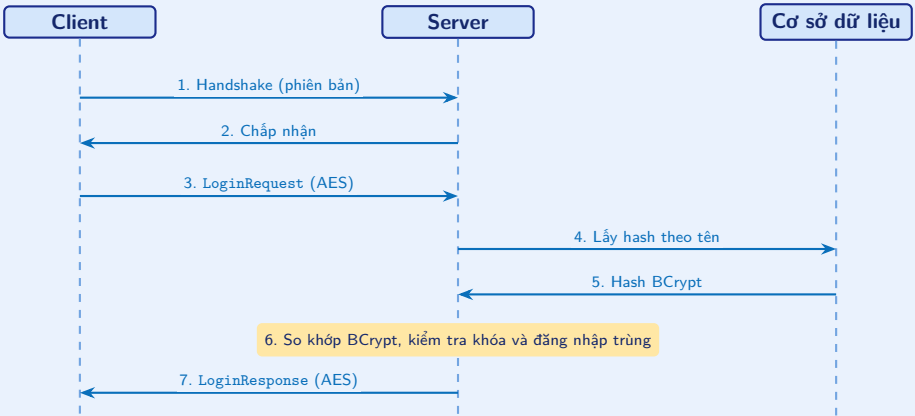
Cấu trúc một gói

`[4 byte độ dài][AES(JSON{Type, Payload})]`

- Type là kiểu liệt kê (PacketType).
- Payload là JSON của DTO tương ứng.
- Gói đầu tiên là Handshake (phiên bản giao thức).

Nhóm	Kiểu gói chính
Tài khoản	LoginRequest/Response, RegisterRequest/Response
Phòng	CreateRoomRequest, JoinRoomRequest/Response, RoomUpdate, LeaveRoom
Ghép trận	MatchmakingJoin/CancelRequest, MatchFound
Trận đấu	PlaceShipsRequest, PlayerReady, GameStart, FireShotRequest, ShotResult, TurnChange, GameOver
Khác	ChatMessage, OnlineListUpdate, Ping/Pong, Error

Luồng Dữ Liệu: Đăng Nhập



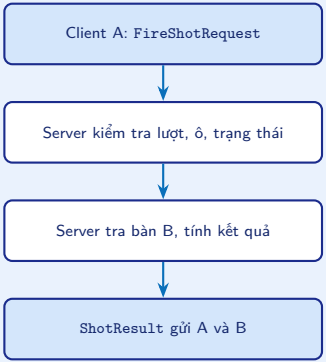
Luồng Dữ Liệu: Một Lượt Bắn (Thiết Kế)

Server-Authoritative

1. Client gửi `FireShotRequest` (tọa độ).
2. Server kiểm tra: **đúng lượt?** ô **chưa bắn?** trận **đang chơi?** Sai thì bỏ qua.
3. Server tra bàn của đối thủ, quyết định **Hit/Miss/Chìm**.
4. Server gửi `ShotResult` cho **cả hai**, rồi `TurnChange`.
5. Hết tàu: gửi `GameOver`, ghi kết quả vào cơ sở dữ liệu.

Đang phát triển

Logic trận trên Server (bước 2–5) chưa xong; xem lịch ở mục Giới thiệu.



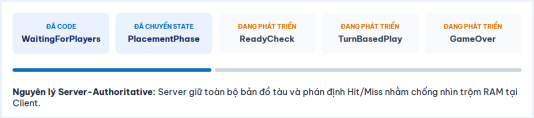
Máy Trạng Thái Của Phòng Và Trận

Các trạng thái

- **Waiting**: chờ đủ 2 người.
- **Placement**: hai bên xếp tàu.
- **Ready**: cả hai bấm sẵn sàng.
- **Playing**: bắn theo lượt.
- **GameOver**: có người thắng hoặc đối thủ mất kết nối.

Quy tắc

Chỉ nhận lệnh bắn khi ở **Playing** và đúng lượt. Bắt đầu trận chỉ khi **cả hai** Ready.



Thiết Kế Cơ Sở Dữ Liệu (ERD)



Một người chơi có nhiều trận; một trận có nhiều lượt bắn (**MatchDetails**). View `leaderboard` chỉ lấy tên, thắng, thua, điểm từ `Users`.

Thiết Kế Bảo Mật Nhiều Lớp

Lớp	Biện pháp	Chống lại
Đường truyền	AES-128-CBC, IV ngẫu nhiên mỗi gói	Nghe lén, đọc trộm gói tin
Mật khẩu	BCrypt (có salt), kiểm tra dù tên không tồn tại	Lộ mật khẩu nếu rò rỉ DB, dò tên tài khoản
Đăng nhập	Khóa 60 giây sau 5 lần sai (theo IP và theo tên)	Dò mật khẩu hàng loạt
Đăng ký	Tối đa 5 lần mỗi 10 phút mỗi IP	Tạo tài khoản rác
Phiên	Router chặn mọi gói (trừ đăng nhập/đăng ký) khi chưa đăng nhập; một tài khoản một phiên	Gửi lệnh khi chưa xác thực, đăng nhập trùng
Dữ liệu	RLS; khóa công khai chỉ đọc view bảng xếp hạng	Đọc trực tiếp bảng `Users` từ ngoài
Luật chơi	Server quyết định Hit/Miss	Client gian lận

Thiết Kế Giao Diện

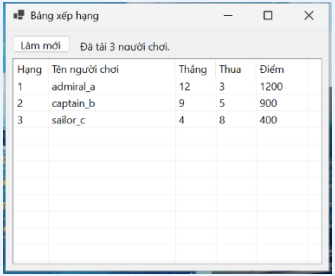
Đăng nhập



Sảnh chờ (Lobby)



Bảng xếp hạng



Đăng ký

[CHƯA CÓ ẢNH]

`images/gk-02-register.png`

Màn hình Đăng ký

Phòng chờ

[CHƯA CÓ ẢNH]

`images/gk-03-roomwait.png`

Phòng chờ có 2 người chơi và nút Ready

Bàn cờ

[CHƯA CÓ ẢNH]

`images/gk-04-board.png`

Lưới 10×10 (chụp từ Client)

Thiết Kế Tính Năng Đặc Biệt

Radar (quét vùng 3x3)

- Mỗi người dùng **một lần mỗi trận**, chọn một ô.
- Server đếm số ô tàu trong vùng 3×3 quanh ô đó và chỉ trả về **con số**, không nói ô nào.
- Dùng Radar thì **mất một lượt**: phải cân nhắc giữa quét và bắn.
- Server giữ cờ "đã dùng" và áp luật đúng lượt: Client không dùng được hai lần.

Gửi kết quả trận ra Discord/Telegram

- Khi trận kết thúc, Server gọi **HTTP POST** tới webhook.
- Tin nhắn xuất hiện ngay trên điện thoại: "A thắng B sau N lượt".
- Nối mạng game (TCP) với Internet (HTTP) và dịch vụ bên ngoài.
- Chạy ở Task riêng với thời hạn 5 giây; lỗi chỉ ghi log, không làm treo Server. URL webhook để trong biến môi trường.

Trạng thái

Đang ở bước thiết kế; hiện thực sau khi Server có logic trận (dự kiến 22/11 – 25/11).